

Requirement Analysis, ER Diagram, Schema Design and UI Design

Database Systems — Group Project (Group 05)

Instructor: Aminul Islam

Group	Project	Date
Group 05	Grocery Management System	September 2026

#	Member Name	Student ID
1	Togor	240207
2	Md. Abu Mursalin	240212

1. Requirement Analysis

Sufian Store is a neighbourhood grocery (mudir doka) at Islamnagar, Gollamari, Khulna. The owner runs everything in paper ledgers: stock is counted by eye, bills are handwritten cash memos, supplier dues are remembered, and monthly profit is guessed. The goal of this project is a simple computer system where the owner, manager and cashier can record products, purchases, sales, customers, suppliers and expenses, and see stock, dues and profit at a glance.

1.1 Questionnaire (shop owner and staff)

We prepared the following questions; answers below are from the shop owner.

Q1. How do you record a sale today?

A: Handwritten cash memo; no copy kept. At day end I count cash in the drawer.

Q2. How do you know when to reorder a product?

A: By looking at the shelf. Sometimes a fast-moving item finishes without notice.

Q3. Where do you buy goods, and how are dues tracked?

A: 2-3 wholesalers (Boro Bazar, Gollamari). Dues are written in a notebook; sometimes pages are lost.

Q4. Do you sell on credit (baki)?

A: Yes, to known customers. I remember who owes how much, or write it on loose slips.

Q5. What expenses besides purchases do you have?

A: Shop rent, electricity, transport/van fare, occasionally staff salary.

Q6. Who will use the computer system?

A: I (owner) see everything; the manager runs daily operations; the cashier only bills at the counter.

Q7. What reports do you need?

A: Today's sales and cash, who still owes money (due), low-stock list, monthly profit.

Q8. Do products expire?

A: Yes — milk, biscuits and oil have expiry dates; expired items are a loss, so I want a warning.

1.2 Interview script (owner, 20 minutes)

Opening: purpose of the project and permission to take notes. Questions: walk me through a normal day from opening to closing; show me the purchase notebook and a cash memo; who are your suppliers and how do you pay them; which products expire or go out of stock most; what would you check first on a computer screen in the morning? Closing: confirm the three user roles and thank the owner.

Summary of findings: (1) billing must be fast — one screen, printable memo; (2) partial payment is normal, so every bill needs cash-received and due; (3) stock must update automatically on purchase and sale; (4) expiry warning within 30 days; (5) cashier must not see profit, purchase cost or expenses.

1.3 Observations (shop visit)

1. Peak hours are morning and evening; billing queues form, so the sale screen keeps items, totals and change/due visible together.
2. The owner checks shelves each morning — matches the low-stock and expiry watch on the dashboard.
3. Supplier vans arrive weekly with a challan (memo) showing item, quantity and rate — matches the purchase entry screen.
4. Regular customers buy on credit and pay later — matches customer records plus per-bill due.

1.4 Business documents studied

5. Paper stock ledger (product, quantity in/out).
6. Handwritten cash memo (date, items, total, cash, signature).
7. Supplier challan/invoice (supplier, date, items with rate).
8. Expense notebook (rent, electricity, transport).

2. Entity-Relationship (ER) Diagram

The diagram below shows all entities, attributes (with data types), primary/foreign/unique keys and relationships of the system.

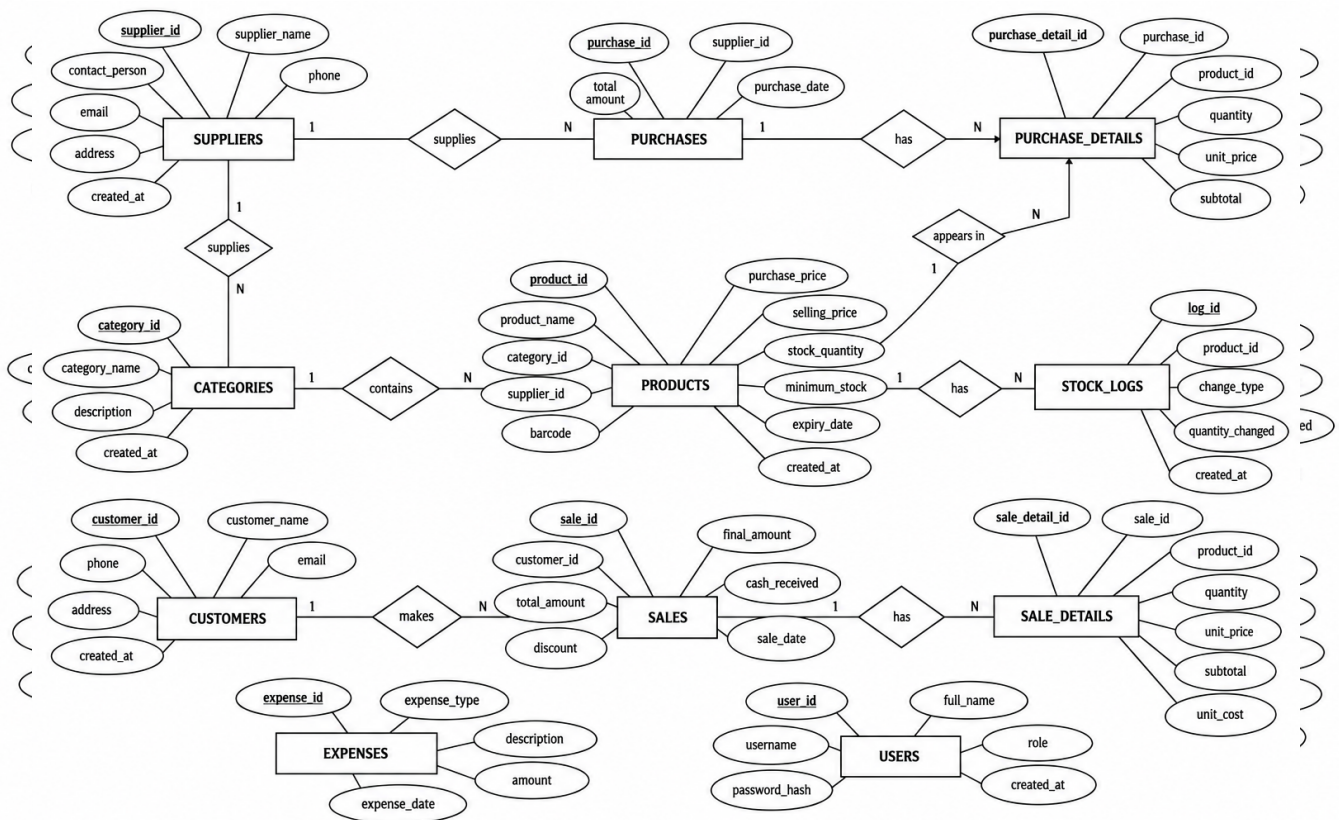


Figure 1: ER diagram — 11 entities with attributes, PK/FK/UNIQUE markers and 1:N relationships (crow's foot = "many"). users and expenses are standalone; the rest form the purchase/sale chain around products.

2.1 Entities and relationships

Relationship	Cardinality	Meaning
categories → products	1 : N	One category holds many products; a product

		belongs to one category.
suppliers → products	1 : N	A supplier is the preferred source of many products.
suppliers → purchases	1 : N	One supplier sends many purchase orders.
purchases → purchase_details → products	M : N (via details)	One purchase contains many products; order lines carry quantity/rate.
customers → sales	1 : N (optional)	A customer has many bills; walk-in sales have no customer.
sales → sale_details → products	M : N (via details)	One bill contains many products; lines carry quantity/price.
products → stock_logs	1 : N	Every stock movement of a product is logged.
users (standalone)	—	Login accounts with roles Owner/Manager/Cashier.
expenses (standalone)	—	Shop running costs (rent, electricity, transport).

3. Schema Design

Primary keys are surrogate integer IDs (AUTO_INCREMENT). Foreign keys enforce referential integrity with ON DELETE RESTRICT for business history (products in bills, suppliers with orders) and SET NULL/CASCADE where safe. Money uses DECIMAL, dates use DATE/DATETIME, roles use ENUM. Two implementation extensions beyond the base diagram: sales.cash_received (partial payment → due = final – cash) and sale_details.unit_cost (purchase-price snapshot at sale time, so later cost changes never rewrite profit history). The schema is in 3NF: every non-key attribute depends only on its table's key (order lines live in detail tables, no repeating groups, no transitive dependencies). Integrity is also enforced with CHECK constraints: prices, stock and amounts ≥ 0; quantity > 0 in purchase_details and sale_details; discount ≤ total_amount; expenses.amount > 0; stock_logs.quantity_changed ≠ 0. products.barcode and users.username are UNIQUE.

Table	Purpose	Primary key	Foreign keys
users	Login accounts + roles	user_id	—
categories	Product groups	category_id	—
suppliers	Wholesalers	supplier_id	—
customers	Credit/regular buyers	customer_id	—
products	Catalog, price, stock, expiry	product_id	category_id, supplier_id
purchases	Purchase order header	purchase_id	supplier_id (NN)
purchase_details	Order lines	purchase_detail_id	purchase_id (NN), product_id (NN)
sales	Bill header + cash received	sale_id	customer_id (nullable)
sale_details	Bill lines + cost snapshot	sale_detail_id	sale_id (NN), product_id (NN)

expenses	Running costs	expense_id	—
stock_logs	Stock movement audit	log_id	product_id (NN)

3.1 Column details (types, keys, constraints)

Table: users

Column	Type	Key	Constraints
user_id	int	PK	AUTO_INCREMENT
username	varchar(100)		NOT NULL, UNIQUE
password_hash	varchar(255)		NOT NULL
full_name	varchar(150)		nullable
role	enum('admin','manager','staff')		NOT NULL, DEFAULT admin
created_at	timestamp		DEFAULT CURRENT_TIMESTAMP

Table: categories

Column	Type	Key	Constraints
category_id	int	PK	AUTO_INCREMENT
category_name	varchar(100)		NOT NULL
description	text		nullable
created_at	timestamp		DEFAULT CURRENT_TIMESTAMP

Table: suppliers

Column	Type	Key	Constraints
supplier_id	int	PK	AUTO_INCREMENT
supplier_name	varchar(150)		NOT NULL
contact_person	varchar(150)		nullable
phone	varchar(20)		nullable
email	varchar(100)		nullable
address	text		nullable
created_at	timestamp		DEFAULT CURRENT_TIMESTAMP

Table: customers

Column	Type	Key	Constraints
customer_id	int	PK	AUTO_INCREMENT
customer_name	varchar(150)		NOT NULL
phone	varchar(20)		nullable
email	varchar(100)		nullable
address	text		nullable
created_at	timestamp		DEFAULT CURRENT_TIMESTAMP

Table: products

Column	Type	Key	Constraints
product_id	int	PK	AUTO_INCREMENT
product_name	varchar(200)		NOT NULL
category_id	int	FK	nullable → categories
supplier_id	int	FK	nullable → suppliers
barcode	varchar(100)		nullable
purchase_price	decimal(10,2)		NOT NULL, DEFAULT 0
selling_price	decimal(10,2)		NOT NULL, DEFAULT 0
stock_quantity	int		NOT NULL, DEFAULT 0
minimum_stock	int		NOT NULL, DEFAULT 5
expiry_date	date		nullable
created_at	timestamp		DEFAULT CURRENT_TIMESTAMP

Table: purchases

Column	Type	Key	Constraints
purchase_id	int	PK	AUTO_INCREMENT
supplier_id	int	FK	NOT NULL → suppliers

total_amount	decimal(12,2)		NOT NULL, DEFAULT 0
purchase_date	datetime		NOT NULL

Table: purchase_details

Column	Type	Key	Constraints
purchase_detail_id	int	PK	AUTO_INCREMENT
purchase_id	int	FK	NOT NULL → purchases
product_id	int	FK	NOT NULL → products
quantity	int		NOT NULL, > 0
unit_price	decimal(10,2)		NOT NULL, ≥ 0
subtotal	decimal(12,2)		NOT NULL (= qty × price)

Table: sales

Column	Type	Key	Constraints
sale_id	int	PK	AUTO_INCREMENT
customer_id	int	FK	nullable → customers (walk-in = NULL)
total_amount	decimal(12,2)		NOT NULL
discount	decimal(10,2)		NOT NULL, DEFAULT 0
final_amount	decimal(12,2)		NOT NULL
cash_received	decimal(12,2)		NOT NULL, ≥ 0; due = final - cash
sale_date	datetime		NOT NULL

Table: sale_details

Column	Type	Key	Constraints
sale_detail_id	int	PK	AUTO_INCREMENT
sale_id	int	FK	NOT NULL → sales
product_id	int	FK	NOT NULL → products
quantity	int		NOT NULL, > 0, ≤ stock

unit_price	decimal(10,2)		NOT NULL, ≥ 0
subtotal	decimal(12,2)		NOT NULL (= qty $\times$ price)
unit_cost	decimal(10,2)		nullable; cost snapshot for COGS

Table: expenses

Column	Type	Key	Constraints
expense_id	int	PK	AUTO_INCREMENT
expense_type	varchar(100)		DEFAULT General
description	text		nullable
amount	decimal(12,2)		NOT NULL, > 0
expense_date	date		NOT NULL
created_at	timestamp		DEFAULT CURRENT_TIMESTAMP

Table: stock_logs

Column	Type	Key	Constraints
log_id	int	PK	AUTO_INCREMENT
product_id	int	FK	NOT NULL $\rightarrow$ products
change_type	varchar(50)		NOT NULL ('purchase'/'sale'/'...')
quantity_changed	int		NOT NULL (+in / -out)
created_at	timestamp		DEFAULT CURRENT_TIMESTAMP

4. UI Design (forms traceable to the ER diagram)

Every form below is a standalone HTML page (folder ui_forms/, open index.html — no server needed) and maps to an entity or relationship of the ER diagram. Foreign keys are chosen from dropdowns (never typed), money uses number inputs with step 0.01, dates use date/datetime inputs, phones follow the Bangladesh mobile format, and required fields, ranges and formats are validated in the browser. On Save each form shows the validated record that would be inserted. (The full project additionally connects these screens to PHP + MySQL.)

Form / page	ER source	Key input controls	Validation / FK handling
user_form.html	users	Text, password, role select	Username unique (letters/digits/_/.); password ≥ 6 chars (stored hashed); role from list
category_form.html	categories	Text, textarea	Name required (2–100 chars)

supplier_form.html	suppliers	Text, phone, email, textarea	Name required; phone 01[3-9]XXXXXXX; valid email format
customer_form.html	customers	Text, phone, email, textarea	Name required; phone/email format checked; walk-ins need no record
product_form.html	products (+ categories, suppliers)	Text, number, date, category & supplier dropdowns	Name required; prices ≥ 0 , stock ≥ 0 whole numbers; FK chosen from dropdown; barcode unique
purchase_form.html	purchases + purchase_details (M:N with products)	Supplier dropdown, repeating product rows (product select, qty, price), datetime	Supplier + ≥ 1 line required; qty ≥ 1 ; subtotal/total auto-calculated; duplicate product blocked
sale_form.html	sales + sale_details (M:N with products)	Customer dropdown (optional), product rows, discount, cash received, datetime	Qty ≥ 1 ; discount $\leq$ total; final and due auto-calculated; walk-in = no customer
expense_form.html	expenses	Type select, date, amount, textarea	Type, date required; amount > 0
stock_adjustment_form.html	stock_logs (+ products)	Product dropdown, change-type radio buttons, quantity	Product FK from list; change type required; quantity non-zero (+ in / - out)

5. How to run and test

- Open ui_forms/index.html in any browser (double-click) to view and test all data-entry forms — no installation needed.
- Database: create the database and import database/schema.sql (structure + constraints), then database/sample_data.sql (3-12 sample rows per table) using phpMyAdmin or the mysql client (MySQL 8.0.16+ / MariaDB 10.2+).
- Optional full application: copy the project (with api/, config/, js/, css/ and the *.html pages) to C:\xampp\htdocs, start Apache + MySQL and open http://localhost/Grocery_Management_System.
- Sample logins: Owner admin / admin123, Manager manager / manager123, Cashier cashier / cashier123.
- Included files: this document, er_diagram.jpg, database/schema.sql, database/sample_data.sql and ui_forms/ (9 forms + index page).